🐍 Python Study Guide

Quantum Software Engineering Role

Core Python Modernization · Unit Testing · Performance Tuning · Exercises

How to Use This Guide

This guide is structured into four sections tailored to your intermediate Python level and target role modernizing quantum control software. Each section includes concepts, before/after code comparisons, practical tips, and hands-on exercises.

Section 1: Core Python Modernization	Section 2: Unit Testing	Section 3: Performance Tuning	Section 4: Exercises
1.1 Type hints & annotations 1.2 Dataclasses & Pydantic 1.3 Pathlib & modern I/O 1.4 Context managers 1.5 Async / await patterns 1.6 Pattern matching	2.1 Why testing matters 2.2 pytest fundamentals 2.3 Parametrize 2.4 pytest.raises 2.5 Fixtures 2.6 Mocking hardware 2.7 Async testing 2.8 Running tests	3.1 Profiling 3.2 NumPy vectorisation 3.3 Numba JIT 3.4 Memory management 3.5 Multiprocessing 3.6 Cython	Week 1 exercises Week 2 exercises Week 3 exercises Week 4 exercises Unit testing exercises Capstone Challenge (standalone section)

Section 1: Core Python Modernization

Legacy quantum control code is often written in older Python styles — no type hints, raw dicts for configuration, and string-based file paths. This section shows you how to modernize such code safely and systematically.

1.1 Type Hints & Annotations

Type hints make code self-documenting and enable static analysis with tools like mypy. They are especially valuable when working with complex data structures like pulse sequences or qubit gate configurations.

Before — Untyped Legacy Code

```
def apply_gate(qubit, gate, params):
    # What is qubit? An int? A string? An object?
    # What shape is params expected to be?
    return run_pulse(qubit, gate, params)
```

After — Modern Typed Code

```
from typing import Sequence
import numpy as np

def apply_gate(
    qubit: int,
    gate: str,
    params: Sequence[float],
) -> np.ndarray:
    """Apply gate to qubit. Returns measurement result array."""
    return run_pulse(qubit, gate, params)
```

> *Run mypy --strict on legacy files to get a prioritised list of typing gaps. Fix them top-down — the type errors in helper functions will ripple through and reveal deeper design issues.*

- Use list[int] instead of List[int] in Python 3.9+
- Use X | None instead of Optional[X] in Python 3.10+
- Use TypeAlias for domain-specific types: QubitIndex = TypeAlias = int
- Add py.typed marker file to mark your package as typed

1.2 Dataclasses & Structured Configuration

Quantum programs often pass around large configuration dictionaries. Replacing these with dataclasses gives you type safety, default values, auto-generated __repr__, and easy serialisation.

Before — Dict-Based Config (Fragile)

```
config = {
    'frequency': 5.1e9,
    'amplitude': 0.5,
    'duration_ns': 20,
}
# Typo 'frequncy' silently creates a new key — no error raised
```

After — Dataclass Config (Safe)

```
from dataclasses import dataclass, field

@dataclass
class PulseConfig:
    frequency: float          # Hz
    amplitude: float          # 0.0 – 1.0
    duration_ns: int = 20
    tags: list[str] = field(default_factory=list)

cfg = PulseConfig(frequency=5.1e9, amplitude=0.5)
print(cfg)  # PulseConfig(frequency=5100000000.0, ...)
```

> *For configs that need validation (e.g., amplitude must be between 0 and 1), consider Pydantic BaseModel instead of dataclasses. Pydantic is widely used in quantum cloud APIs like IBM Quantum.*

1.3 Pathlib — Modern File Paths

The os.path module is still common in legacy code but is verbose and error-prone. pathlib.Path provides an elegant object-oriented interface.

Before — os.path style

```
import os
base = '/data/experiments'
out = os.path.join(base, 'run_42', 'results.json')
os.makedirs(os.path.dirname(out), exist_ok=True)
```

After — pathlib style

```
from pathlib import Path

base = Path('/data/experiments')
out = base / 'run_42' / 'results.json'
out.parent.mkdir(parents=True, exist_ok=True)
out.write_text(json.dumps(results))  # one-liner write
```

1.4 Context Managers

Quantum hardware interfaces — instrument connections, file handles, shared locks — need reliable cleanup. Context managers guarantee teardown even when exceptions occur.

Writing a Custom Context Manager

```
from contextlib import contextmanager

@contextmanager
def instrument_session(address: str):
    instr = connect(address)
    try:
        yield instr
    finally:
        instr.close()  # always runs

with instrument_session('GPIB::5') as instr:
    instr.send_pulse(cfg)
```

> *Always use context managers when dealing with hardware connections. A crashed experiment should never leave an instrument in an undefined state.*

1.5 Async / Await Patterns

Quantum experiments often involve waiting — for measurements, for instrument acknowledgments, for cloud job results. Async/await lets you run multiple jobs concurrently without threads.

Concurrent Job Submission

```
import asyncio

async def run_experiment(qubit_id: int) -> dict:
    result = await hardware.submit_and_wait(qubit_id)
    return result

async def run_all(qubits: list[int]) -> list[dict]:
    tasks = [run_experiment(q) for q in qubits]
    return await asyncio.gather(*tasks)  # runs concurrently

results = asyncio.run(run_all([0, 1, 2, 3]))
```

- Use asyncio.gather() for concurrent I/O-bound tasks (network, hardware polling)
- Use asyncio.Queue for producer-consumer pipelines (e.g., data acquisition)
- Avoid async for CPU-bound tasks — use multiprocessing instead

1.6 Pattern Matching (Python 3.10+)

Pattern matching is ideal for parsing gate operations, handling different result types from quantum backends, and routing commands in control software.

Gate Dispatch with match/case

```
def execute_gate(gate: dict) -> np.ndarray:
    match gate:
        case {'type': 'X', 'qubit': int(q)}:
            return apply_x(q)
        case {'type': 'RZ', 'qubit': int(q), 'angle': float(theta)}:
            return apply_rz(q, theta)
        case {'type': 'CNOT', 'control': int(c), 'target': int(t)}:
            return apply_cnot(c, t)
        case _:
            raise ValueError(f'Unknown gate: {gate}')
```

Pattern matching replaces long if/elif chains in command parsers and makes adding new gate types much cleaner. It is structurally similar to Haskell pattern matching if you have an FP background.

Section 2: Unit Testing

Quantum control software is safety-critical — a miscalibrated pulse can damage a qubit. Unit tests are the primary defence against regressions when modernising legacy code. This section covers the essential pytest patterns used in scientific Python projects.

2.1 Why Testing Matters in Quantum Software

When you refactor a 3,000-line pulse sequencer, how do you know you haven't changed its behaviour? Tests give you a safety net that lets you modernise aggressively without fear. They also serve as executable documentation — a test named test_rz_gate_produces_correct_phase tells you exactly what the code must do.

Testing Goal	Tool	When to Use
Correctness	`pytest + assert`	*All functions, always*
Numerical accuracy	`np.testing.assert_allclose`	*Floating-point & array results*
Edge cases	`pytest.mark.parametrize`	*Multiple inputs, boundary values*
Expected failures	`pytest.raises`	*Validation, error paths*
Hardware mocking	`unittest.mock / pytest-mock`	*Instrument I/O, cloud APIs*
Async functions	`pytest-asyncio`	*async def coroutines*
Performance guards	`pytest-benchmark`	*Catch speed regressions*

2.2 pytest Fundamentals

pytest is the standard test runner in scientific Python. It requires no boilerplate — just name your file test_*.py and your functions test_*() and pytest finds them automatically.

Basic Test Structure

```
# test_gates.py
import numpy as np
import pytest
from my_gates import rz_matrix, PulseConfig

def test_rz_identity_at_zero():
    """RZ(0) should be the identity matrix."""
    result = rz_matrix(0.0)
    np.testing.assert_allclose(result, np.eye(2, dtype=complex), atol=1e-10)

def test_rz_half_pi_phase():
    """RZ(π/2) diagonal entries should have magnitude 1."""
    m = rz_matrix(np.pi / 2)
```

```
    assert abs(abs(m[0, 0]) - 1.0) < 1e-10
    assert abs(abs(m[1, 1]) - 1.0) < 1e-10
```

> *Always use np.testing.assert_allclose instead of == for floating-point comparisons. Floating-point equality is almost never exact. Set atol (absolute) and rtol (relative) tolerances appropriate to your domain.*

2.3 Parametrize — Testing Multiple Inputs Cleanly

Instead of copying a test five times for five input values, use @pytest.mark.parametrize to run the same test logic across many cases with a single declaration.

```
@pytest.mark.parametrize('theta,expected_trace', [
    (0.0,        2.0),   # identity — trace = 2
    (np.pi,      0.0),   # Z gate — trace = 0 (off-diagonal)
    (np.pi / 2,  2.0),   # S gate — trace still 2
    (2 * np.pi,  2.0),   # full rotation — back to identity
])
def test_rz_trace(theta: float, expected_trace: float):
    m = rz_matrix(theta)
    np.testing.assert_allclose(m.trace().real, expected_trace, atol=1e-10)
```

2.4 pytest.raises — Testing Error Paths

Validation logic in dataclasses and functions must be tested too. Use pytest.raises as a context manager to assert that the right exception is raised with the right message.

```
def test_pulse_config_rejects_invalid_amplitude():
    with pytest.raises(ValueError, match='amplitude must be between'):
        PulseConfig(frequency_hz=5.1e9, amplitude=1.5, duration_ns=20)

def test_pulse_config_rejects_negative_duration():
    with pytest.raises(ValueError, match='duration_ns must be positive'):
        PulseConfig(frequency_hz=5.1e9, amplitude=0.5, duration_ns=-1)
```

2.5 Fixtures — Reusable Test Setup

Fixtures provide shared setup and teardown for tests. They are the pytest replacement for setUp/tearDown in unittest. Common uses: creating standard gate configs, mock instrument objects, or pre-built NumPy arrays.

```
import pytest
import numpy as np

@pytest.fixture
def standard_pulse_config():
    """A valid PulseConfig used across multiple tests."""
```

```
    return PulseConfig(frequency_hz=5.1e9, amplitude=0.5, duration_ns=20)

@pytest.fixture
def random_angles():
    """100 random angles in [-2π, 2π] for property-based style tests."""
    rng = np.random.default_rng(seed=42)  # fixed seed for reproducibility
    return rng.uniform(-2 * np.pi, 2 * np.pi, size=100)

def test_config_default_tags(standard_pulse_config):
    assert standard_pulse_config.tags == []

def test_rz_unitarity(random_angles):
    """RZ must be unitary for all angles: U @ U† = I."""
    matrices = rz_matrices_fast(random_angles)  # shape (100, 2, 2)
    identity = np.eye(2, dtype=complex)
    for m in matrices:
        np.testing.assert_allclose(m @ m.conj().T, identity, atol=1e-10)
```

> *Use a fixed random seed in fixtures (np.random.default_rng(seed=42)) so tests are reproducible. Flaky tests that pass sometimes and fail other times are worse than no tests.*

2.6 Mocking Hardware — unittest.mock

Quantum tests cannot run against real hardware in a CI pipeline. Use unittest.mock.MagicMock or pytest-mock's mocker fixture to replace instrument calls with fake objects that record interactions.

```
from unittest.mock import MagicMock, patch

def test_instrument_session_calls_close_on_exception():
    mock_instr = MagicMock()

    with patch('my_module.connect', return_value=mock_instr):
        with pytest.raises(RuntimeError):
            with instrument_session('GPIB::5'):
                raise RuntimeError('Experiment failed')

    mock_instr.close.assert_called_once()  # disconnect must have run
```

2.7 Testing Async Code with pytest-asyncio

Async functions require a special test runner. Install pytest-asyncio and mark async tests with @pytest.mark.asyncio.

```
pip install pytest-asyncio
```

```
import pytest
import asyncio
```

```
@pytest.mark.asyncio
async def test_concurrent_readout_returns_all_results():
    results = await run_all(qubits=[0, 1, 2, 3])
    assert len(results) == 4
    assert all('qubit' in r and 'result' in r for r in results)

@pytest.mark.asyncio
async def test_concurrent_readout_is_fast():
    start = asyncio.get_event_loop().time()
    await run_all(qubits=list(range(5)))
    elapsed = asyncio.get_event_loop().time() - start
    assert elapsed < 1.0, f'Expected <1s, got {elapsed:.2f}s'
```

2.8 Running Tests — Key Commands

Command	What it does
`pytest`	Run all tests
`pytest -v`	Verbose — show each test name
`pytest -x`	Stop after first failure
`pytest -k 'rz'`	Run only tests matching 'rz'
`pytest --tb=short`	Shorter traceback output
`pytest --cov=src --cov-report=term`	Coverage report in terminal
`pytest -n 4`	Parallel test run (pytest-xdist)

Section 3: Performance Tuning

Quantum control software must often operate in near-real-time, processing large arrays of pulse data, running calibration loops, or simulating circuit fidelity. Knowing where and how to speed up Python is a core skill for this role.

3.1 Profiling — Find the Bottleneck First

Never optimise blindly. Always profile first. The bottleneck is almost never where you expect it.

cProfile — Function-Level Profiling

```
python -m cProfile -s cumtime my_script.py | head -30
```

line_profiler — Line-by-Line Timing

```
pip install line_profiler

@profile  # decorator added by kernprof
def calibrate_qubit(qubit_id: int) -> float:
    data = acquire_shots(qubit_id, n=1000)
    freq = np.fft.rfft(data)           # <- is this slow?
    return np.argmax(np.abs(freq))

kernprof -l -v calibrate.py
```

> *The workflow is always: Profile → identify hot line → optimise → re-profile. Repeat until fast enough. A 10x speedup in a line that takes 0.1% of runtime saves you nothing.*

3.2 NumPy Vectorisation

Quantum data is inherently numerical — shot arrays, Bloch vectors, density matrices, pulse envelopes. Python loops over NumPy arrays are 10–100x slower than vectorised operations.

Before — Python Loop (Slow)

```
# Calculate Hamming weight of 10,000 measurement bitstrings
weights = []
for bits in measurement_strings:   # Python loop: slow
    weights.append(bin(bits).count('1'))
```

After — Vectorised (Fast)

```
import numpy as np
```

```
# popcount via bit tricks — operates on entire array at once
arr = np.array(measurement_strings, dtype=np.uint32)
weights = np.unpackbits(arr.view(np.uint8), axis=-1).sum(axis=-1)
```

Pattern	Use When	Typical Speedup
`np.vectorize / broadcasting`	Element-wise ops on arrays	**10–50x**
`np.einsum`	Tensor contractions (gate matrices)	**2–10x**
`np.fft.rfft / scipy.signal`	Spectral analysis of IQ data	**100x+ vs pure Python**

3.3 Numba — JIT Compilation

Numba compiles Python functions to machine code at runtime. It is ideal for numerical loops that are hard to vectorise but need to be fast — for example, custom gate fidelity estimators or iterative calibration algorithms.

Numba Example — RB Fidelity Loop

```
from numba import njit
import numpy as np

@njit(cache=True)  # cache=True avoids recompiling next run
def rb_decay(shots: np.ndarray, depths: np.ndarray) -> np.ndarray:
    """Randomised benchmarking survival probability per depth."""
    result = np.empty(len(depths))
    for i, d in enumerate(depths):
        mask = shots[:, 0] == d
        result[i] = shots[mask, 1].mean()
    return result
```

- @njit is faster than @jit because it disables Python fallback
- cache=True saves compiled machine code to disk — avoids cold-start cost
- Numba works best with NumPy arrays and basic Python arithmetic
- Avoid Python objects, dicts, or dynamic types inside @njit functions

3.4 Memory Management — Arrays & Views

In quantum data pipelines, memory copies are a hidden performance killer. NumPy views let you slice and reshape arrays without allocating new memory.

Views vs Copies

```
data = np.random.randn(1_000_000)

# VIEW — zero copy, shares memory with 'data'
```

```
view = data[::2]           # every other element
view = data.reshape(1000, 1000)

# COPY — new allocation
copy = data[::2].copy()
copy = np.ascontiguousarray(data[::2])  # needed before Numba
```

Memory Layout — C vs Fortran Order

```
# Quantum gate matrices — row-major (C order) for row access
gate = np.eye(4, dtype=complex, order='C')

# Column access is faster with Fortran order
state = np.zeros((4, n_shots), dtype=complex, order='F')
```

> *Use np.shares_memory(a, b) to check if two arrays share memory. Unexpected copies are a common source of memory bloat in long experiments.*

3.5 Multiprocessing & Concurrency

Python's Global Interpreter Lock (GIL) prevents true CPU parallelism in threads. For CPU-bound tasks like circuit simulation across many qubits, use multiprocessing instead.

ProcessPoolExecutor — Parallel Circuit Evaluation

```
from concurrent.futures import ProcessPoolExecutor
import numpy as np

def simulate_circuit(params: tuple[float, ...]) -> float:
    """Simulate one circuit, return fidelity."""
    ...

param_grid = [(0.1, 0.5), (0.2, 0.5), (0.3, 0.5)]

with ProcessPoolExecutor(max_workers=8) as pool:
    fidelities = list(pool.map(simulate_circuit, param_grid))
```

Tool	Best For	GIL Bypassed?
`threading`	I/O-bound tasks, hardware polling	No
`asyncio`	Concurrent awaitable I/O	No
`multiprocessing`	CPU-bound simulation/processing	**Yes**
`ProcessPoolExecutor`	Easy parallel map over data	**Yes**

3.6 Cython — When You Need C-Level Speed

When Numba is not sufficient (e.g., for low-latency real-time loops or interfacing with C instrument drivers), Cython compiles Python to C extension modules.

Cython — Typed Loop Example

```
# file: fast_rb.pyx
cimport numpy as cnp
import numpy as np

def apply_decay(
    cnp.ndarray[cnp.float64_t, ndim=1] data,
    double decay_rate
) -> cnp.ndarray:
    cdef int n = data.shape[0]
    cdef int i
    cdef cnp.ndarray[cnp.float64_t, ndim=1] out = np.empty(n)
    for i in range(n):
        out[i] = data[i] * (1.0 - decay_rate) ** i
    return out
```

> *Prefer Numba over Cython when possible — it requires no compilation step or build system. Use Cython only when you need to interface with C libraries or when Numba's constraints are too limiting.*

Section 4: Practice Exercises

Each exercise is grounded in the quantum software engineering context. They are ordered by week and build on each other — complete them in sequence for best results. For each exercise, write your solution, then check it manually against the hints provided.

Week 1 Exercises — Core Modernization

Exercise 1.1 — Add Type Hints to a Legacy Gate Runner

The function below is untyped. Add complete type annotations and a docstring. Use Sequence, np.ndarray, and a TypeAlias for QubitId.

```
# STARTER CODE
def run_gate(qubit, gate_name, angles, shots):
    results = []
    for i in range(shots):
        r = execute(qubit, gate_name, angles)
        results.append(r)
    return results
```

Your goal	Define QubitId = TypeAlias = int. Annotate all parameters and return type. Run mypy --strict on your file and fix all errors.
Hint	angles is a Sequence[float], return type is list[np.ndarray]. Import TypeAlias from typing (Python 3.10+) or use a comment-style alias.

Exercise 1.2 — Refactor a Config Dict to a Dataclass

Refactor the dictionary below into a typed dataclass. Add validation logic using __post_init__ to ensure amplitude stays within 0.0–1.0 and duration_ns is positive.

```
# STARTER CODE
pulse_config = {
    'frequency_hz': 5.1e9,
    'amplitude': 0.85,
    'duration_ns': 40,
    'backend': 'ibm_kyoto',
    'tags': ['calibration', 'q0'],
}
```

Your goal	Create @dataclass PulseConfig with typed fields, default tags=[], and __post_init__ raising ValueError for invalid amplitude or duration_ns.
Hint	Use field(default_factory=list) for the tags list. In __post_init__, check: if not 0.0 <= self.amplitude <= 1.0: raise ValueError(...)

Exercise 1.3 — Context Manager for Instrument Connection

Write a context manager using @contextmanager that simulates connecting to and disconnecting from a lab instrument. It should print connection/disconnection messages and yield a mock instrument object. Test it handles exceptions cleanly.

```
# STARTER CODE — fill in the implementation
from contextlib import contextmanager

@contextmanager
def instrument_session(address: str):
    # TODO: simulate connect, yield, disconnect
    ...

# Test it:
with instrument_session('GPIB::5') as instr:
    print(f'Sending pulse via {instr}')
    raise RuntimeError('Experiment failed!')  # disconnect must still run
```

Your goal	The disconnect message must print even when RuntimeError is raised. Verify this is the case.
Hint	Wrap the yield in try/finally. The finally block runs regardless of exceptions. Yield a simple string like f'Instrument@{address}' as the mock object.

Week 2 Exercises — Async & Pattern Matching

Exercise 2.1 — Concurrent Qubit Readout with asyncio

Simulate reading out 5 qubits concurrently. Each readout takes a random delay (0.1–0.5s). Use asyncio.gather() to run all readouts in parallel and print total elapsed time.

```
# STARTER CODE
import asyncio, random, time

async def readout_qubit(qubit_id: int) -> dict:
    # TODO: await asyncio.sleep for random 0.1–0.5s delay
    # Return {'qubit': qubit_id, 'result': random 0 or 1}
    ...

async def main() -> None:
    # TODO: gather readouts for qubits 0..4
    # Print results and total elapsed time
    ...

asyncio.run(main())
```

Your goal	Total elapsed time should be ~0.5s maximum (not ~2.5s), proving concurrency is working. Confirm this in your output.
Hint	tasks = [readout_qubit(i) for i in range(5)]; results = await asyncio.gather(*tasks). Wrap main() call with time.perf_counter() before and after asyncio.run().

Exercise 2.2 — Gate Dispatcher with Pattern Matching

Write a function execute_instruction(instr: dict) that uses match/case to dispatch quantum gate instructions. Handle X, RZ, CNOT, and MEASURE gates. Raise ValueError for unknown gates.

```
# Test cases your dispatcher must handle:
execute_instruction({'op': 'X',     'qubit': 0})
execute_instruction({'op': 'RZ',    'qubit': 1, 'theta': 1.5708})
execute_instruction({'op': 'CNOT',  'control': 0, 'target': 1})
execute_instruction({'op': 'MEASURE','qubit': 0, 'classical_bit': 0})
execute_instruction({'op': 'UNKNOWN'})  # must raise ValueError
```

Your goal	Each case should print a descriptive string (e.g., 'Applying X to qubit 0'). Bonus: add a guard clause to reject theta values outside [-2π, 2π].
Hint	case {'op': 'RZ', 'qubit': int(q), 'theta': float(t)} if abs(t) <= 2*math.pi: ... Use case _ as the final catch-all.

Week 3 Exercises — Profiling & NumPy Vectorisation

Exercise 3.1 — Profile and Identify a Bottleneck

Run the slow script below through cProfile, identify the top bottleneck, then use line_profiler to pinpoint the exact line. Document your findings.

```
# STARTER CODE — deliberately slow
import numpy as np, time

def compute_expectation(shots: list[list[int]]) -> float:
    total = 0.0
    for shot in shots:               # Python loop — slow
        for bit in shot:
            total += bit             # scalar accumulation
    return total / (len(shots) * len(shots[0]))

shots = [[int(b) for b in format(i % 256, '08b')] for i in range(50_000)]
print(compute_expectation(shots))
```

Your goal	Profile with cProfile. Then write compute_expectation_fast() using np.array and np.mean. Benchmark both versions with timeit and record the speedup.

Hint	Fast version: arr = np.array(shots); return arr.mean(). Expect 50–100x speedup. Use python -m timeit -s 'from script import shots, f' 'f(shots)' for both versions.

Exercise 3.2 — Vectorise a Rotation Matrix Batch

The loop below computes a batch of single-qubit RZ rotation matrices. Rewrite it using NumPy broadcasting to eliminate the Python loop entirely.

```
# STARTER CODE — Python loop
import numpy as np

def rz_matrices_slow(angles: list[float]) -> list[np.ndarray]:
    matrices = []
    for theta in angles:
        m = np.array([
            [np.exp(-1j * theta / 2), 0],
            [0,  np.exp( 1j * theta / 2)],
        ])
        matrices.append(m)
    return matrices
```

Your goal	Write rz_matrices_fast(angles: np.ndarray) -> np.ndarray returning shape (N, 2, 2). No Python loops. Verify np.allclose(slow_result[i], fast_result[i]) for all i.
Hint	phases = np.exp(1j * angles / 2) gives shape (N,). Build result = np.zeros((N,2,2), dtype=complex); result[:,0,0] = phases.conj(); result[:,1,1] = phases.

Week 4 Exercises — Numba, Memory & Parallelism

Exercise 4.1 — Accelerate a Calibration Loop with Numba

Apply @njit to the function below, which calculates T1 decay fidelity at each time step. Benchmark the JIT-compiled version against the original NumPy version.

```
# STARTER CODE
import numpy as np
from numba import njit

# Baseline: pure NumPy
def t1_decay_numpy(times: np.ndarray, t1: float) -> np.ndarray:
    return np.exp(-times / t1)

# TODO: implement with @njit — iterate manually in a loop
# @njit(cache=True)
def t1_decay_numba(times: np.ndarray, t1: float) -> np.ndarray:
    ...
```

Your goal	Write the @njit version using an explicit for loop. Warm it up with one call, then benchmark. Note: for simple np.exp NumPy may win — the learning goal is the @njit workflow, not the speedup result.
Hint	result = np.empty(len(times)); for i in range(len(times)): result[i] = math.exp(-times[i] / t1). Import math inside @njit — it compiles to libm calls.

Exercise 4.2 — Parallel Parameter Sweep

A calibration sweep tests 20 different drive amplitudes across qubits 0–3. Write a parallel version using ProcessPoolExecutor that runs all 80 combinations concurrently across CPU cores.

```
# STARTER CODE
import time, random
from concurrent.futures import ProcessPoolExecutor
from itertools import product

def calibrate(qubit: int, amplitude: float) -> dict:
    time.sleep(random.uniform(0.05, 0.15))  # simulate hardware
    fidelity = 1.0 - abs(amplitude - 0.5) * 0.4 + random.gauss(0, 0.01)
    return {'qubit': qubit, 'amplitude': amplitude, 'fidelity': fidelity}

amplitudes = [round(0.1 + i * 0.04, 2) for i in range(20)]
params = list(product(range(4), amplitudes))  # 80 combinations

# TODO: run calibrate(*p) for all p in params using ProcessPoolExecutor
# Print the (qubit, amplitude) pair with highest fidelity
```

Your goal	The parallel version should complete in ~0.15s, not ~9s (80 * 0.1s serial). Verify timing. Then find and print the best (qubit, amplitude) combination by fidelity.
Hint	pool.map() requires a single-argument function — use a wrapper: def _run(p): return calibrate(*p). Then: results = list(pool.map(_run, params)). Use max(results, key=lambda r: r['fidelity']).

Capstone Challenge

Exercise 4.3 — Full Refactor Challenge

This is the final integrating exercise. The script below represents a typical piece of legacy quantum control code. Your task is to fully modernise and optimise it, applying everything from this guide.

```
# LEGACY CODE — modernise and optimise this
import numpy as np

def run_experiment(config, qubit_list, n_shots):
    results = {}
    for q in qubit_list:
        shots = []
        for i in range(n_shots):
            angle = config['angle']
            amp   = config['amplitude']
            r = np.sin(angle * amp + q * 0.1)  # mock result
            shots.append(r)
        results[q] = sum(shots) / len(shots)
    return results

cfg   = {'angle': 1.57, 'amplitude': 0.8, 'backend': 'sim'}
qubits = list(range(8))
out   = run_experiment(cfg, qubits, 10_000)
```

Checklist	
	• Replace config dict with a typed @dataclass with validation • Add full type hints to all function signatures • Eliminate both Python loops using NumPy broadcasting • Make the qubit sweep run in parallel with ProcessPoolExecutor • Verify output is identical to the original using np.allclose() • Benchmark: the modernised version should be at least 20x faster

Section 5: Unit Testing Exercises

These exercises build a complete test suite for the code you wrote in Sections 1–2. By the end you will have a test file covering correctness, validation, parametrized cases, hardware mocking, and async behaviour.

Exercise 5.1 — Test a Typed Gate Function

Write three unit tests for the rz_matrix() function you created (or use the version from Exercise 1.1). Cover: identity at zero, correct diagonal phases at π, and unitarity for an arbitrary angle.

```
# STARTER CODE — test_rz.py
import numpy as np
import pytest
# from your_module import rz_matrix

def test_rz_zero_is_identity():
    ...  # TODO

def test_rz_pi_flips_phase():
    ...  # TODO: diagonal should be [-1j, 1j] (up to global phase)

def test_rz_is_unitary():
    ...  # TODO: m @ m.conj().T == identity for theta = 0.7
```

Your goal	All three tests must pass with pytest -v. Use np.testing.assert_allclose with atol=1e-10 throughout — never use == on complex arrays.
Hint	Unitarity check: np.testing.assert_allclose(m @ m.conj().T, np.eye(2, dtype=complex), atol=1e-10). RZ(π) diagonals: exp(±iπ/2) = ±i.

Exercise 5.2 — Parametrize Across Multiple Angles

Extend your test suite with a parametrized test that verifies the trace of rz_matrix(theta) equals 2*cos(theta/2) for five different angles including boundary values.

```
# Add to test_rz.py
@pytest.mark.parametrize('theta', [
    0.0, np.pi / 4, np.pi / 2, np.pi, 2 * np.pi
])
def test_rz_trace_formula(theta: float):
    ...  # TODO: verify trace == 2 * cos(theta / 2)
```

Your goal	pytest should report 5 individual test cases (one per angle). Run with -v to confirm each is listed separately.
Hint	m = rz_matrix(theta); trace = m.trace().real; expected = 2 * np.cos(theta / 2). Then

	assert_allclose(trace, expected, atol=1e-10).

Exercise 5.3 — Validate Dataclass Errors with pytest.raises

Write two tests that confirm your PulseConfig dataclass (from Exercise 1.2) correctly rejects invalid inputs. Also write one test confirming valid inputs are accepted without error.

```
# test_pulse_config.py
import pytest
# from your_module import PulseConfig

def test_valid_config_is_accepted():
    ...  # TODO: should not raise

def test_amplitude_above_one_raises():
    ...  # TODO: amplitude=1.1 must raise ValueError

def test_negative_duration_raises():
    ...  # TODO: duration_ns=-5 must raise ValueError
```

Your goal	Use pytest.raises(ValueError, match='...') with a regex that matches your error message. This ensures the right error is raised for the right reason.
Hint	with pytest.raises(ValueError, match='amplitude'): PulseConfig(..., amplitude=1.1, ...). The match argument is a regex checked against str(exception).

Exercise 5.4 — Mock a Hardware Instrument

Write a test that verifies your instrument_session() context manager (from Exercise 1.3) calls close() on the instrument even when an exception occurs mid-experiment. Use unittest.mock to avoid needing real hardware.

```
# test_instrument.py
from unittest.mock import MagicMock, patch
import pytest
# from your_module import instrument_session

def test_instrument_close_called_on_exception():
    mock_instr = MagicMock()
    # TODO: patch 'your_module.connect' to return mock_instr
    # TODO: enter instrument_session, raise RuntimeError inside
    # TODO: assert mock_instr.close was called exactly once
    ...

def test_instrument_close_called_on_success():
    mock_instr = MagicMock()
    # TODO: same but without raising — close must still be called
    ...
```

Your goal	Both tests must pass without connecting to any real instrument. mock_instr.close.assert_called_once() should pass in both cases.
Hint	with patch('your_module.connect', return_value=mock_instr): with pytest.raises(RuntimeError): with instrument_session('GPIB::5'): raise RuntimeError('fail'). Then assert close was called.

Exercise 5.5 — Test an Async Function with pytest-asyncio

Write two async tests for your concurrent qubit readout function from Exercise 2.1: one verifying the result structure, and one verifying it completes in under 1 second for 5 qubits.

```
# test_readout.py
import pytest, asyncio
# from your_module import run_all

@pytest.mark.asyncio
async def test_readout_returns_one_result_per_qubit():
    results = await run_all([0, 1, 2, 3, 4])
    ...  # TODO: assert length and structure

@pytest.mark.asyncio
async def test_readout_completes_concurrently():
    import time
    start = time.perf_counter()
    await run_all([0, 1, 2, 3, 4])
    elapsed = time.perf_counter() - start
    ...  # TODO: assert elapsed < 1.0
```

Your goal	Install pytest-asyncio and add asyncio_mode = 'auto' to pytest.ini or pyproject.toml. Both tests should pass and the timing test confirms concurrency is real.
Hint	assert len(results) == 5; assert all('qubit' in r and 'result' in r for r in results). For timing: assert elapsed < 1.0, f'Took {elapsed:.2f}s — is asyncio.gather being used?'

Quick Reference: Tools & Commands

Task	Command / Tool	Notes
Type check	`mypy --strict src/`	*Fix errors top-down*
Lint / format	`ruff check . && ruff format .`	*Modern replacement for flake8+black*
Profile (function)	`python -m cProfile -s cumtime`	*Sort by cumulative time*
Profile (line)	`kernprof -l -v script.py`	*Requires @profile decorator*
Memory usage	`memray run script.py`	*Best Python memory profiler*
Benchmark	`python -m timeit -n 1000 'expr'`	*Quick timing of expressions*
Array info	`a.dtype, a.shape, a.flags`	*Check contiguity for Numba*
Numba compile	`@njit(cache=True)`	*Cache saves recompile time*
Parallel map	`ProcessPoolExecutor.map()`	*CPU-bound tasks*
Async gather	`asyncio.gather(*tasks)`	*I/O-bound concurrent tasks*
Run tests	`pytest -v`	*Verbose test output*
Run with coverage	`pytest --cov=src`	*Coverage report*

4-Week Study Plan

This plan is designed to take you from reviewing concepts to confidently applying modern Python in a quantum software engineering context.

Week	Focus Areas	Daily Goals	Milestone
Week 1	**Core Modernization**	• Days 1-2: Type hints + mypy on a legacy file • Days 3-4: Refactor dicts to dataclasses • Day 5: Pathlib + context managers	*Modernised script: fully typed, dataclass configs, pathlib I/O, safe resource handling*
Week 2	**Unit Testing**	• Days 1-2: Write tests for typed functions with pytest • Days 3-4: Add parametrize, raises, and fixture tests • Day 5: Mock hardware and test async readout	*Complete test suite covering correctness, validation, mocking, and async behaviour*
Week 3	**Profiling & NumPy**	• Days 1-2: Profile a slow script with cProfile + line_profiler • Days 3-4: Vectorise identified loops with NumPy • Day 5: Benchmark before vs after	*Documented profiling report + vectorised rewrite with measured speedup*
Week 4	**Numba, Memory & Parallelism**	• Days 1-2: Apply @njit to a numerical loop • Days 3-4: ProcessPoolExecutor for parallel sweep • Day 5: Full integration — modernise + tune a complete script	*Complete refactor: typed, tested, profiled and optimised codebase*

Good luck with the role!

The quantum computing software stack is moving fast. Engineers who can bridge classical Python best practices with hardware-aware performance optimisation are highly sought after. This guide gives you the foundation — come back often and extend it as you grow.